

习题解答 HW5-2

TA-艾琮焜



9.3 9.3 对图 9.32 的流图，计算；

(a) 为到达-定值分析，计算每个块的 gen 、 $kill$ 、IN 和 OUT 集合。

(b) 为可用表达式分析，计算每个块的 e_gen 、 e_kill 、IN 和 OUT 集合。

(c) 为活跃变量分析，计算每个块的 def 、 use 、IN 和 OUT 集合。

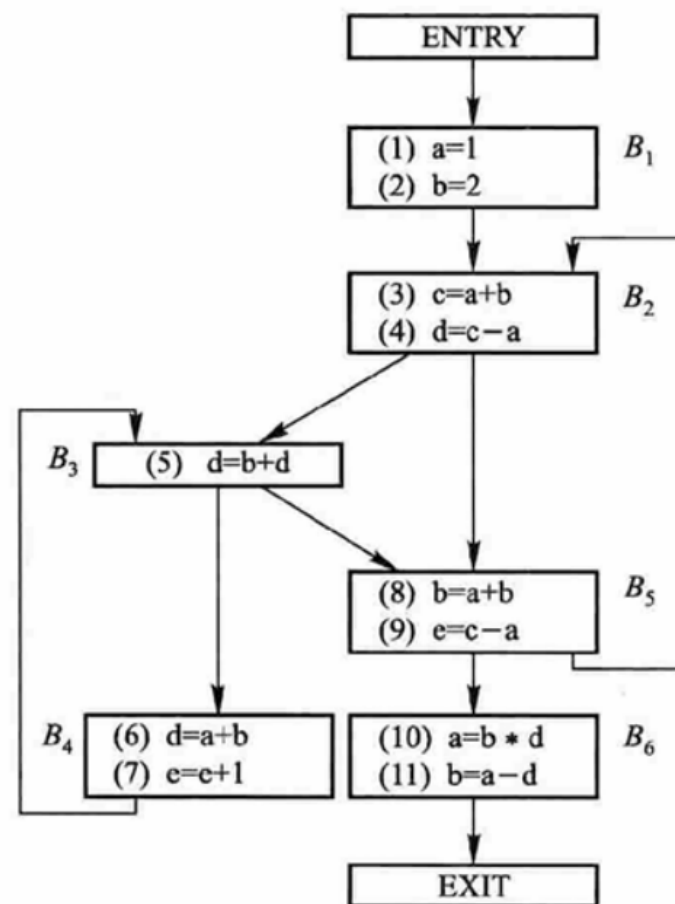


图 9.32 一个流图

9.3(a) 为到达-定值分析，计算每个块的 gen 、 $kill$ 、IN 和 OUT 集合。

• 到达-定值的数据流等式

- gen_B : B 中能到达 B 的结束点的定值语句
- $kill_B$: 整个程序中决不会到达 B 结束点的定值
- $IN[B]$: 能到达 B 的开始点的定值集合
- $OUT[B]$: 能到达 B 的结束点的定值集合

OUT集合计算方法：迭代直到不动点

$$IN[B] = \bigcup_{P \text{ 是 } B \text{ 的前驱}} OUT[P]$$

$$OUT[B] = gen_B \cup (IN[B] - kill_B)$$

$$OUT[ENTRY] = \emptyset$$

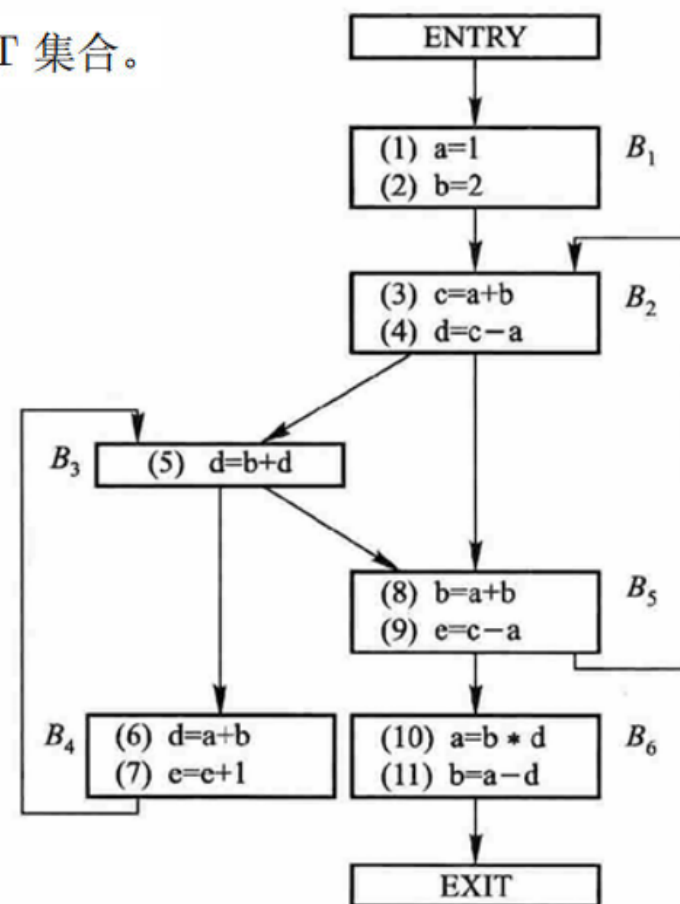


图 9.32 一个流图

9.3(a) 为到达-定值分析，计算每个块的 *gen*、*kill*、IN 和 OUT 集合。

用11位二进制数表示11个表达式是否在集合中，
1表示在，0表示不在，从左至右表示表达式(1) ~ (11)

每个基本块的*gen*, *kill*集合

基本块	B1	B2	B3	B4	B5	B6
gen	(1), (2)	(3), (4)	(5)	(6), (7)	(8), (9)	(10), (11)
kill	(8), (10), (11)	(5), (6)	(4), (6)	(4), (5), (9)	(2), (7), (11)	(1), (2), (8)

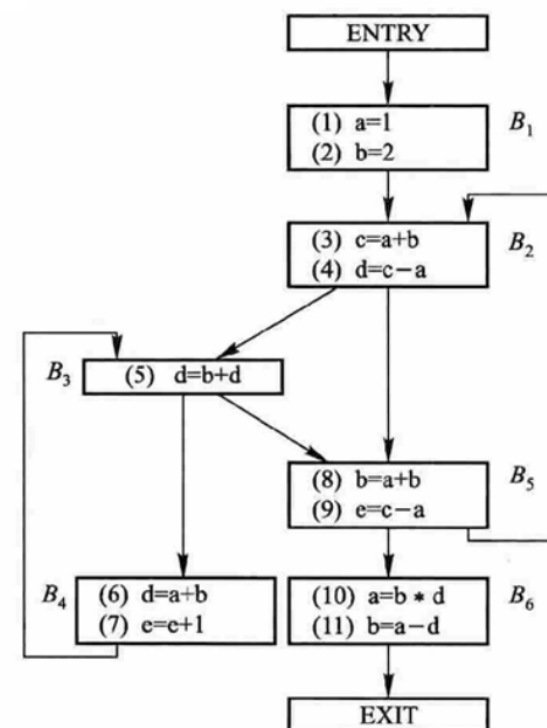


图 9.32 一个流图

9.3(a) 为到达-定值分析，计算每个块的 *gen*、*kill*、IN 和 OUT 集合。

用11位二进制数表示11个表达式是否在集合中，
1表示在，0表示不在，从左至右表示表达式(1) ~ (11)

第一次迭代结束

基本块	B1	B2	B3	B4	B5	B6
IN	000 0000 0000	110 0000 0000	111 1000 0000	111 0100 0000	111 1100 0000	101 1100 1100
OUT	110 0000 0000	111 1000 0000	111 0100 0000	111 0011 0000	101 1100 1100	001 1100 0111

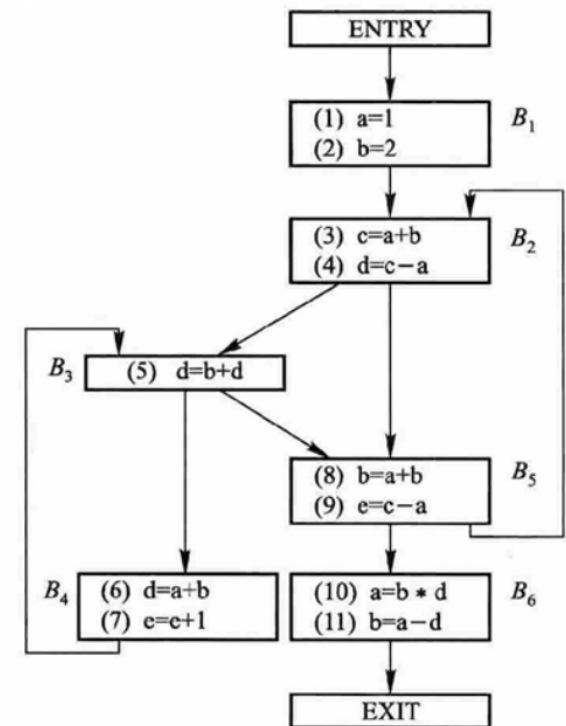


图 9.32 一个流图

9.3(a) 为到达-定值分析，计算每个块的 *gen*、*kill*、IN 和 OUT 集合。

用11位二进制数表示11个表达式是否在集合中，
1表示在，0表示不在，从左至右表示表达式(1) ~ (11)

第二次迭代结束

基本块	B1	B2	B3	B4	B5	B6
IN	000 0000 0000	111 1100 1100	111 1011 1100	111 0101 1100	111 1101 1100	101 1100 1100
OUT	110 0000 0000	111 1000 1100	111 0101 1100	111 0011 1000	101 1100 1100	001 1100 0111

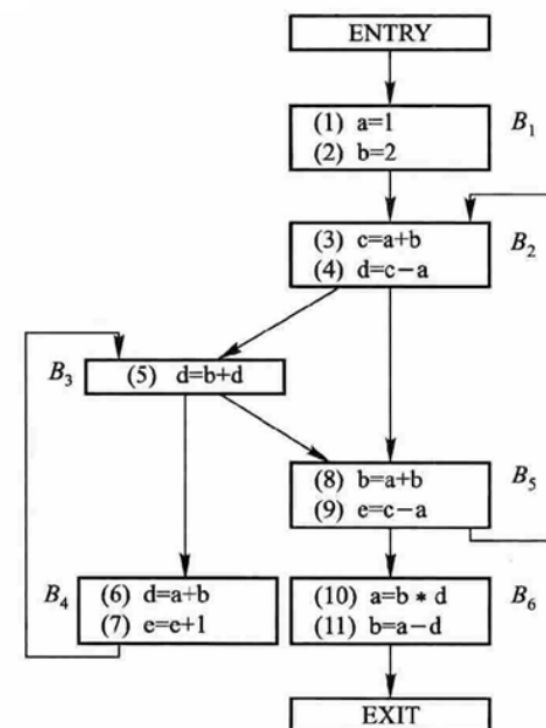


图 9.32 一个流图

9.3(a) 为到达-定值分析，计算每个块的 *gen*、*kill*、IN 和 OUT 集合。

用11位二进制数表示11个表达式是否在集合中，
1表示在，0表示不在，从左至右表示表达式(1) ~ (11)

第三次迭代结束，迭代结果不变，停止迭代

基本块	B1	B2	B3	B4	B5	B6
IN	000 0000 0000	111 1100 1100	111 1011 1100	111 0101 1100	111 1101 1100	101 1100 1100
OUT	110 0000 0000	111 1000 1100	111 0101 1100	111 0011 1000	101 1100 1100	001 1100 0111

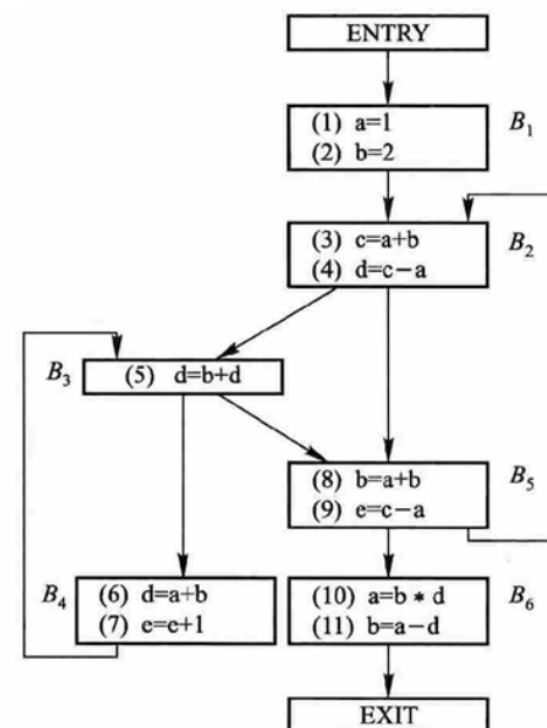


图 9.32 一个流图

9.3(b) 为可用表达式分析, 计算每个块的 e_gen 、 e_kill 、IN 和 OUT 集合。

- e_gen_B : 块 B 产生的可用表达式集合
- e_kill_B : 块 B 注销的可用表达式集合
- $IN[B]$: 块 B 入口的可用表达式集合
- $OUT[B]$: 块 B 出口的可用表达式集合

OUT集合计算方法: 迭代直到不动点

$$OUT[B] = e_gen_B \cup (IN[B] - e_kill_B)$$

$$IN[B] = \cap_{P \text{ 是 } B \text{ 的前驱}} OUT[P]$$

$$OUT[ENTRY] = \emptyset$$

- 在ENTRY的出口处没有可用表达式

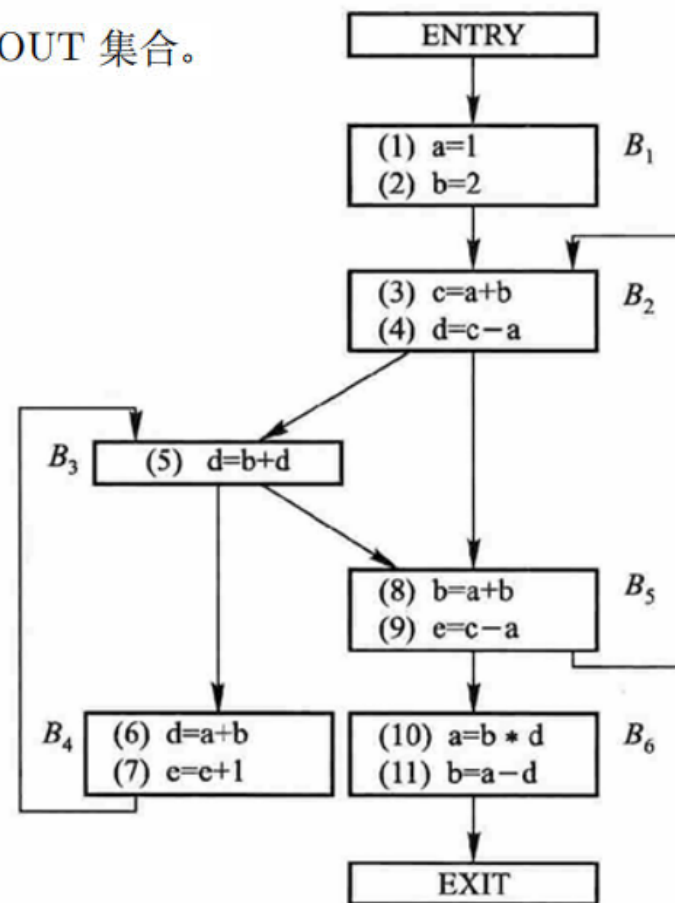


图 9.32 一个流图

9.3(b) 为可用表达式分析, 计算每个块的 e_gen 、 e_kill 、IN 和 OUT 集合。

全部表达式 $U = \{1, 2, a+b, c-a, b+d, e+1, b*d, a-d\}$

基本块	B1	B2	B3	B4	B5	B6
e_gen	$\{1, 2\}$	$\{a+b, c-a\}$	$\emptyset$	$\{a+b\}$	$\{c-a\}$	$\{a-d\}$
e_kill	$\{a+b, c-a, b+d, b*d, a-d\}$	$\{b+d, b*d, a-d\}$	$\{b+d, b*d, a-d\}$	$\{b+d, e+1, b*d, a-d\}$	$\{a+b, b+d, e+1, b*d\}$	$\{a+b, c-a, b+d, b*d\}$

$e_gen[ENTRY] = e_gen[EXIT]$

$= e_kill[ENTRY] = e_kill[EXIT] = \emptyset$

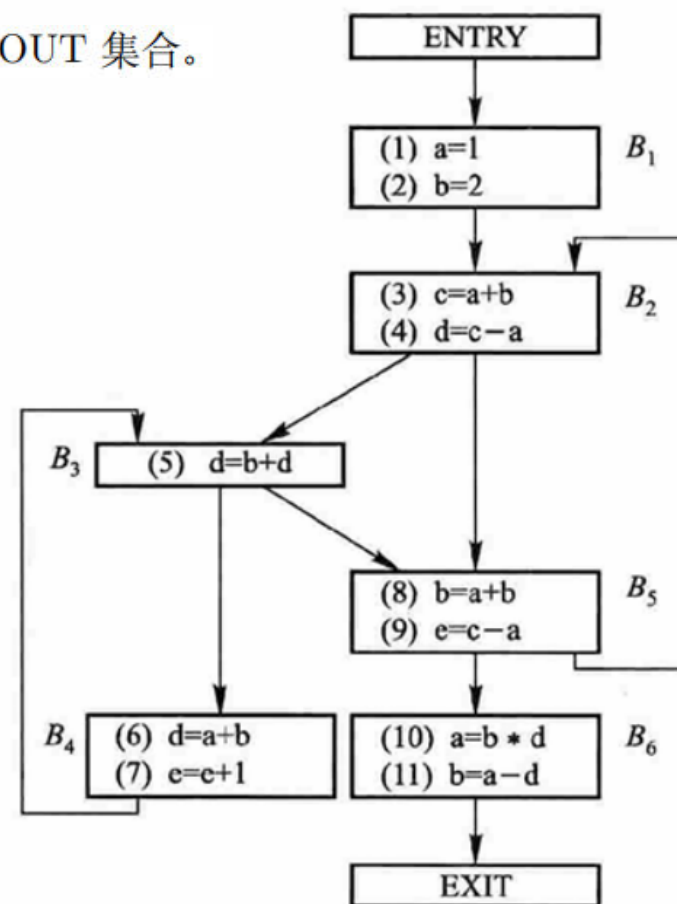


图 9.32 一个流图

9.3(b) 为可用表达式分析，计算每个块的 e_gen 、 e_kill 、IN 和 OUT 集合。

迭代开始前：

基本块	B1	B2	B3	B4	B5	B6
IN						
OUT	U	U	U	U	U	U

$OUT[ENTRY] = \emptyset$

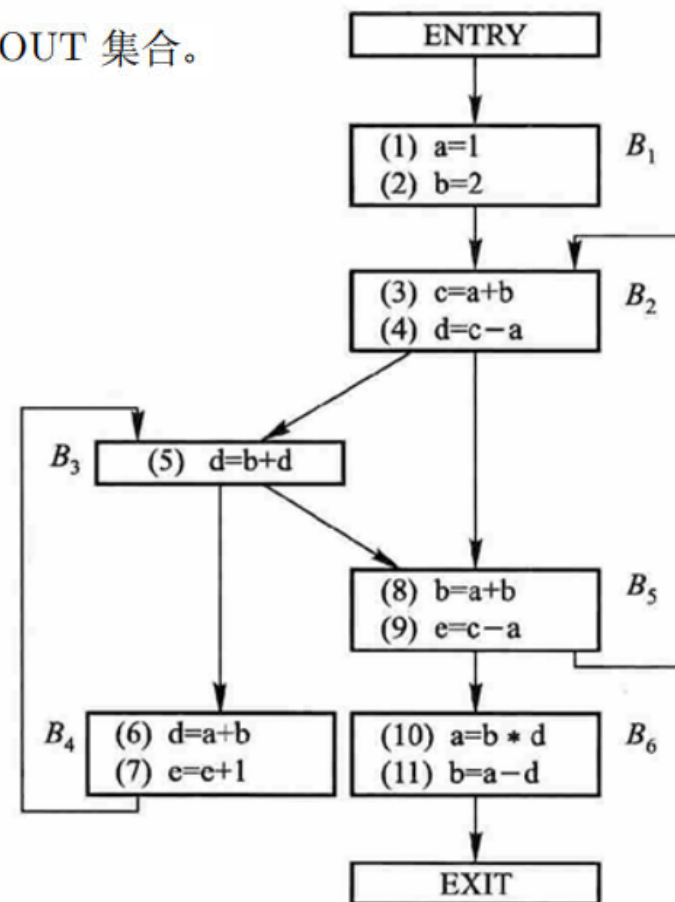


图 9.32 一个流图

9.3(b) 为可用表达式分析, 计算每个块的 e_gen 、 e_kill 、IN 和 OUT 集合。

第一次迭代结束:

基本块	B1	B2	B3	B4	B5	B6
IN	$\emptyset$	{1, 2}	{1, 2, a+b, c-a}	{1, 2, a+b, c-a}	{1, 2, a+b, c-a}	{1, 2, c-a}
OUT	{1, 2}	{1, 2, a+b, c-a}	{1, 2, a+b, c-a}	{1, 2, a+b, c-a}	{1, 2, c-a}	{1, 2, a-d}

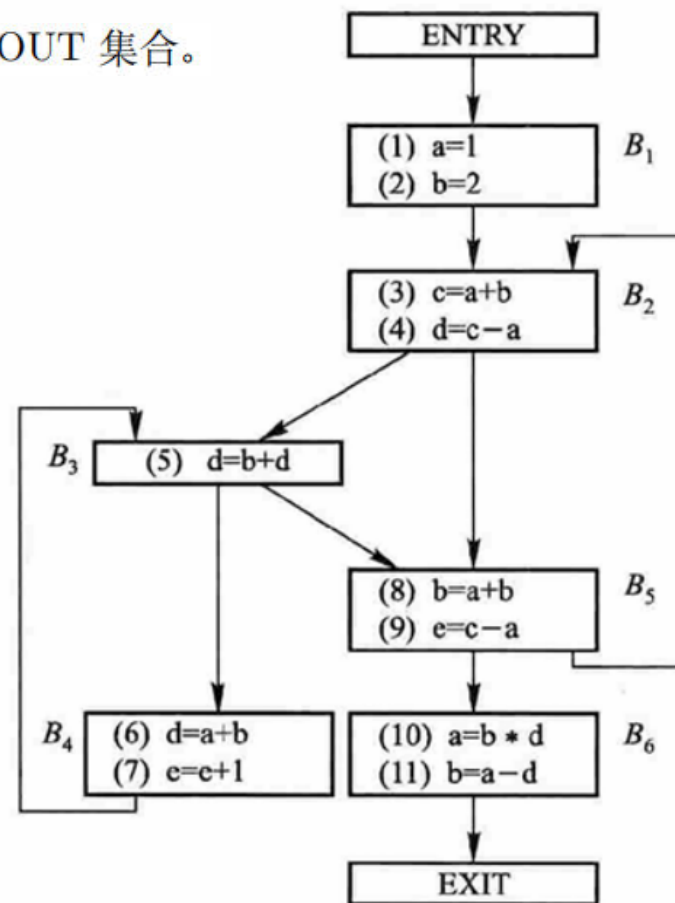


图 9.32 一个流图

9.3(b) 为可用表达式分析，计算每个块的 e_gen 、 e_kill 、IN 和 OUT 集合。

第二次迭代结束：迭代结果不变，停止迭代

基本块	B1	B2	B3	B4	B5	B6
IN	$\emptyset$	$\{1, 2\}$	$\{1, 2, a+b, c-a\}$	$\{1, 2, a+b, c-a\}$	$\{1, 2, a+b, c-a\}$	$\{1, 2, c-a\}$
OUT	$\{1, 2\}$	$\{1, 2, a+b, c-a\}$	$\{1, 2, a+b, c-a\}$	$\{1, 2, a+b, c-a\}$	$\{1, 2, c-a\}$	$\{1, 2, a-d\}$

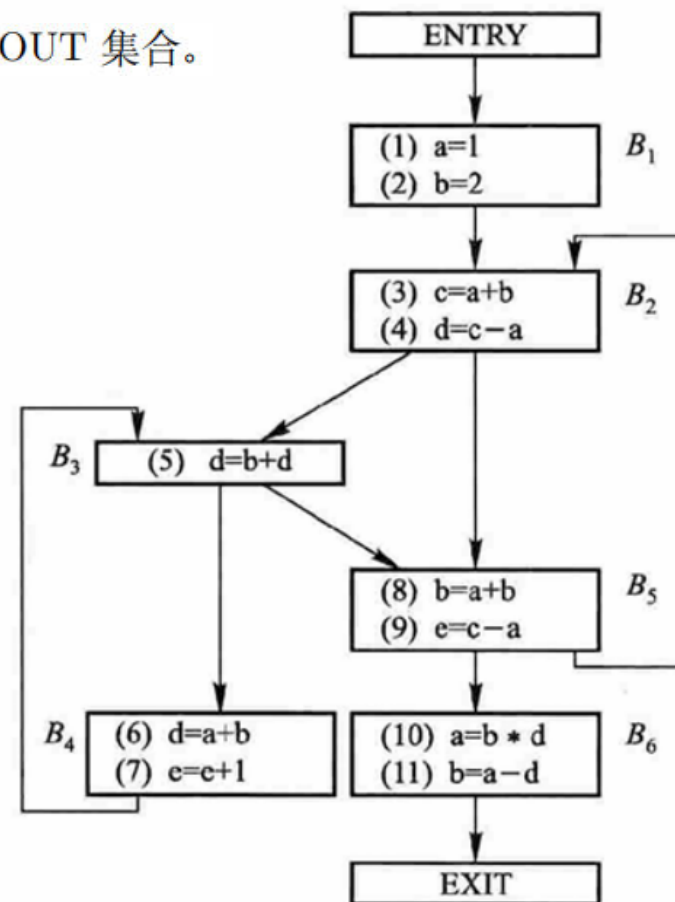


图 9.32 一个流图

9.3(c) 为活跃变量分析，计算每个块的 def 、 use 、IN 和 OUT 集合。

- $IN[B]$: 块 B 开始点的活跃变量集合
- $OUT[B]$: 块 B 结束点的活跃变量集合
- use_B : 块 B 中有引用，且在引用前在 B 中没有被定值的变量集
- def_B : 块 B 中有定值，且该定值前在 B 中没有被引用的变量集

IN集合计算方法：迭代直到不动点

$$IN[EXIT] = \emptyset$$

- 边界条件：程序出口处没有活跃变量

$$OUT[B] = \bigcup_{S \text{ 是 } B \text{ 的后继}} IN[S]$$

$$IN[B] = use_B \cup (OUT[B] - def_B)$$

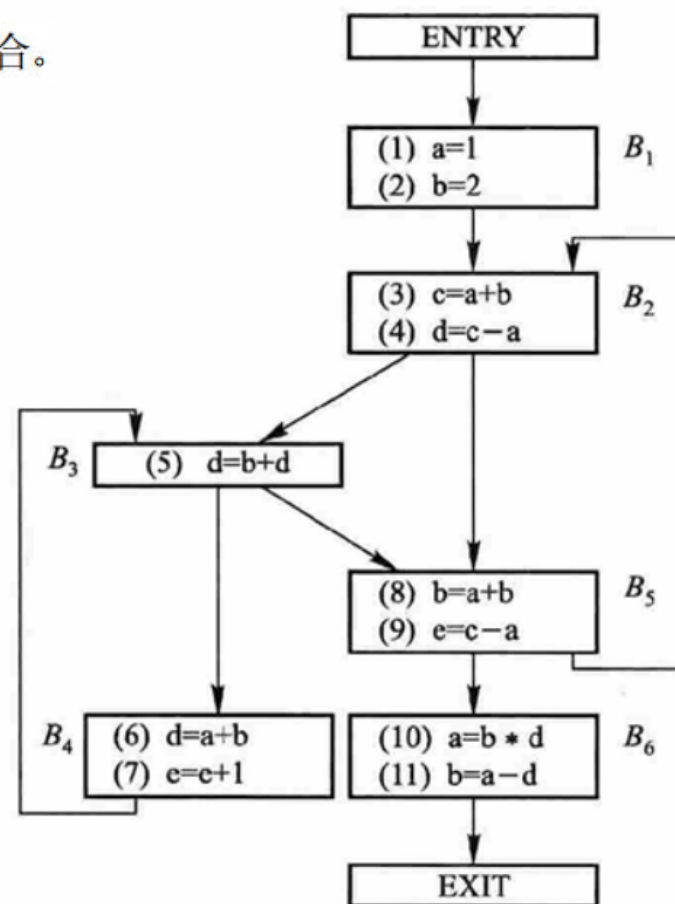


图 9.32 一个流图

9.3(c) 为活跃变量分析，计算每个块的 *def*、*use*、IN 和 OUT 集合。

基本块	B1	B2	B3	B4	B5	B6
use	$\emptyset$	{a,b}	{b,d}	{a,b,e}	{a,b,c}	{b,d}
def	{a,b}	{c,d}	$\emptyset$	{d}	{e}	{a}

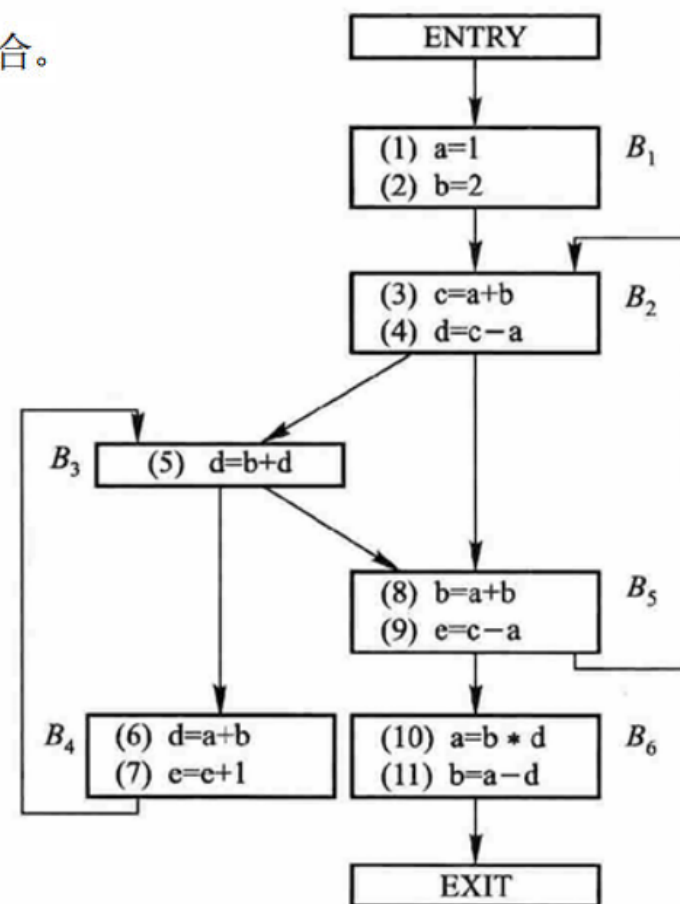


图 9.32 一个流图

9.3(c) 为活跃变量分析，计算每个块的 *def*、*use*、IN 和 OUT 集合。

迭代开始前：

基本块	B1	B2	B3	B4	B5	B6
IN	$\emptyset$	$\emptyset$	$\emptyset$	$\emptyset$	$\emptyset$	$\emptyset$
OUT						

$IN[EXIT] = \emptyset$

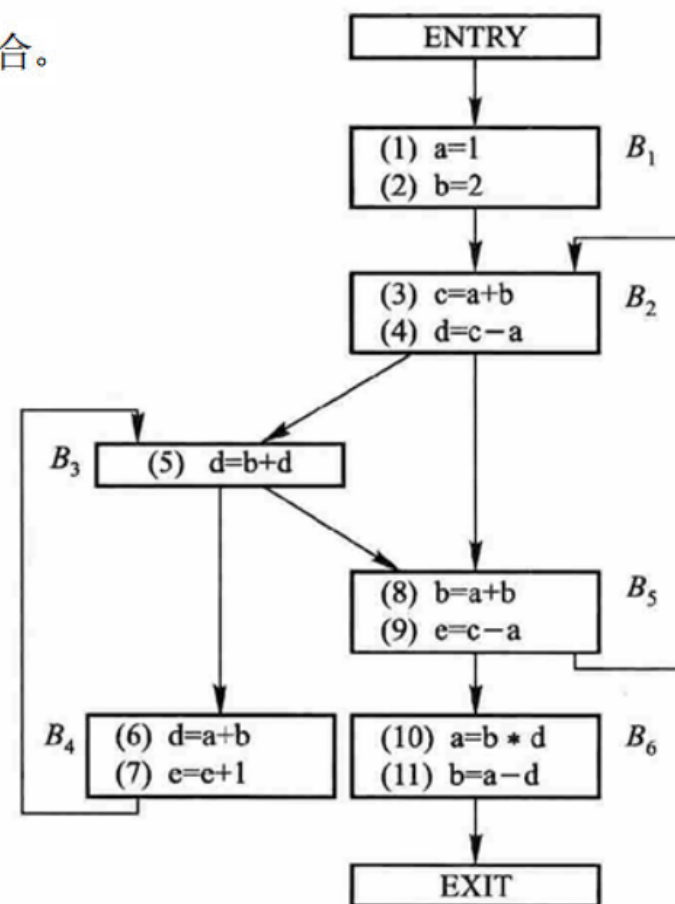


图 9.32 一个流图

9.3(c) 为活跃变量分析，计算每个块的 *def*、*use*、IN 和 OUT 集合。

第一次迭代结束：

基本块	B1	B2	B3	B4	B5	B6
IN	{e}	{a,b,e}	{a,b,c,d,e}	{a,b,e}	{a,b,c,d}	{b,d}
OUT	{a,b,e}	{a,b,c,d,e}	{a,b,c,d,e}	$\emptyset$	{b,d}	$\emptyset$

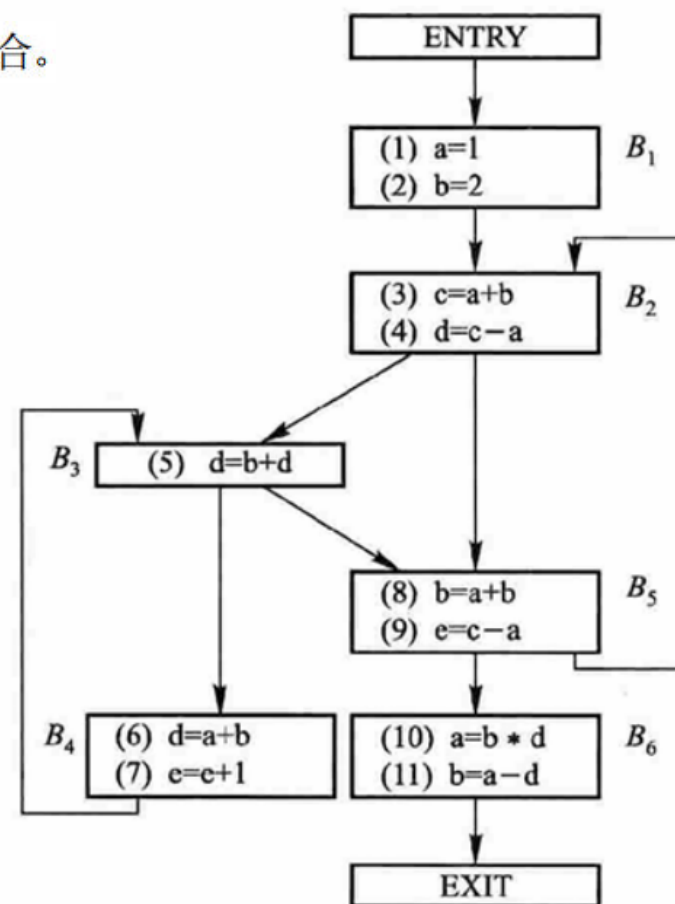


图 9.32 一个流图

9.3(c) 为活跃变量分析，计算每个块的 *def*、*use*、IN 和 OUT 集合。

第二次迭代结束：

基本块	B1	B2	B3	B4	B5	B6
IN	{e}	{a,b,e}	{a,b,c,d,e}	{a,b,c,e}	{a,b,c,d}	{b,d}
OUT	{a,b,e}	{a,b,c,d,e}	{a,b,c,d,e}	{a,b,c,d,e}	{a,b,d,e}	$\emptyset$

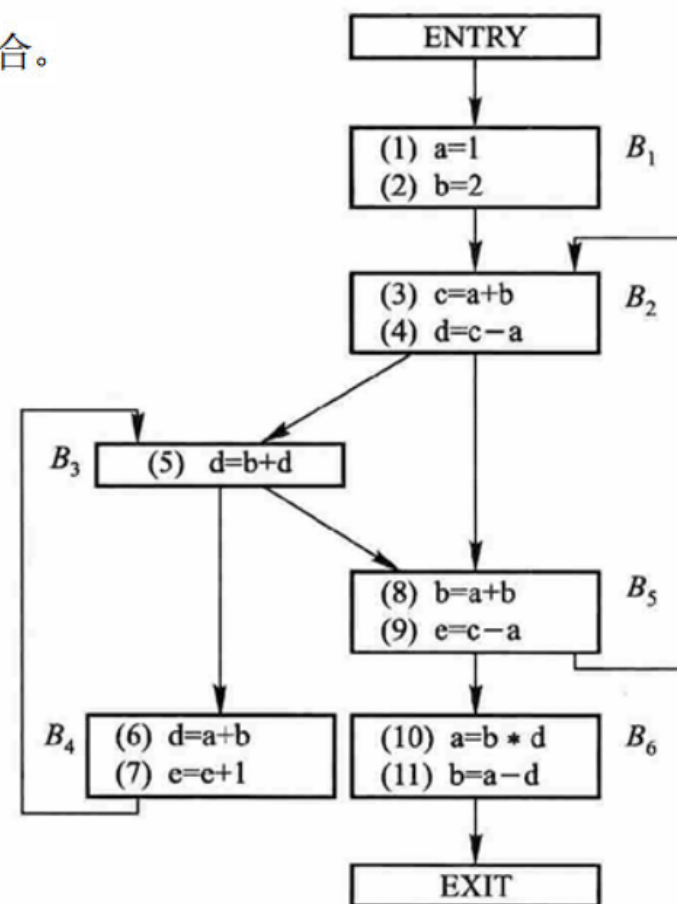


图 9.32 一个流图

9.3(c) 为活跃变量分析，计算每个块的 *def*、*use*、IN 和 OUT 集合。

第三次迭代结束：迭代结果不变，停止迭代

基本块	B1	B2	B3	B4	B5	B6
IN	{e}	{a,b,e}	{a,b,c,d,e}	{a,b,c,e}	{a,b,c,d}	{b,d}
OUT	{a,b,e}	{a,b,c,d,e}	{a,b,c,d,e}	{a,b,c,d,e}	{a,b,d,e}	$\emptyset$

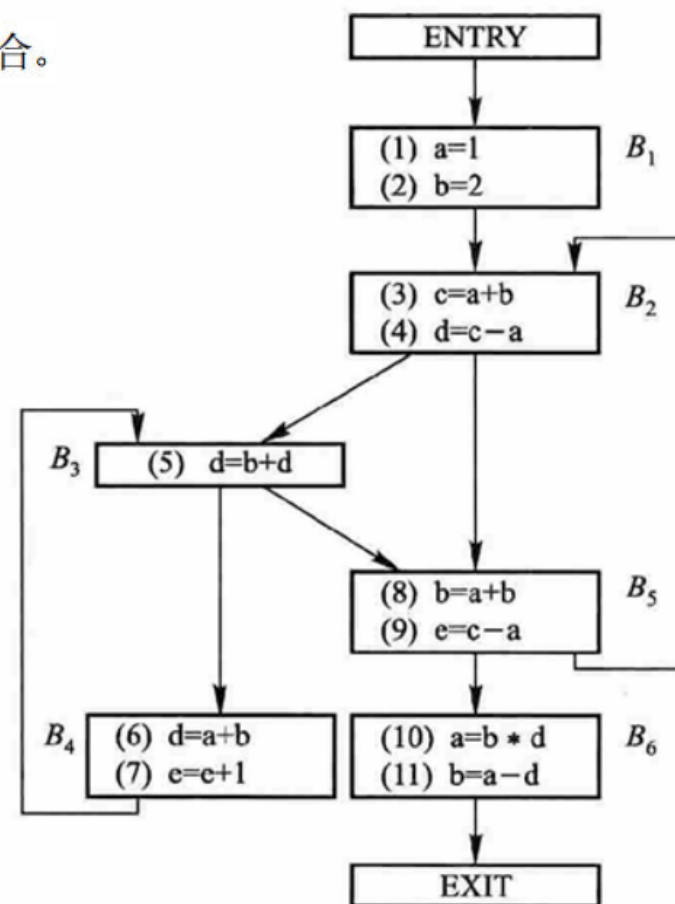


图 9.32 一个流图

9.15(a)

9.15 对图 9.32 的流图：

(a) 计算支配关系。

(f) 找出该流图的自然循环。

流图中结点d支配n：从流图起点开始每条到达n的路径都要经过d，写成 $d \text{ dom } n$ ，称d是n的支配结点。

每个结点支配自身。

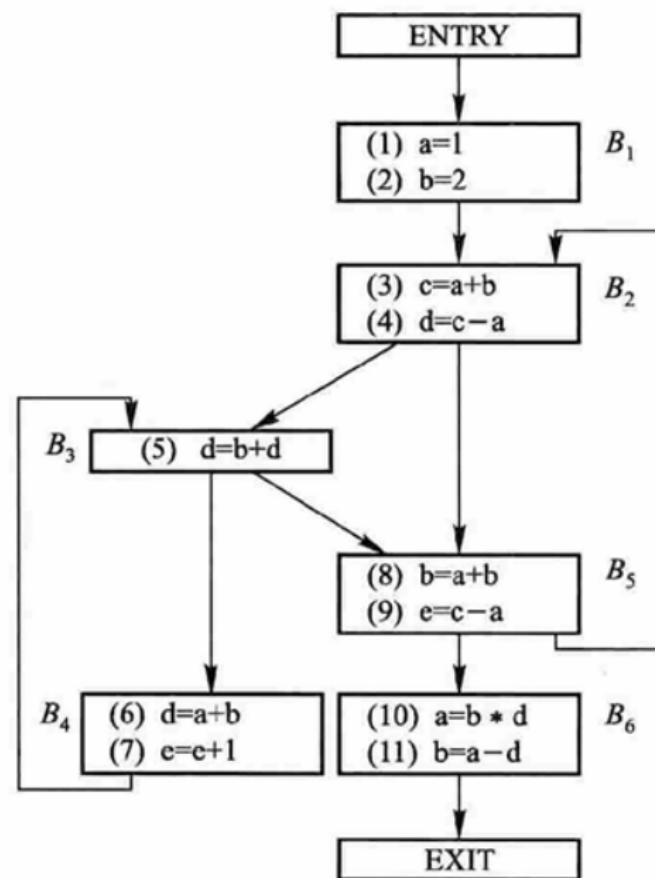


图 9.32 一个流图

9.15(a)

- 计算支配关系:

支配结点	支配对象
1	1, 2, 3, 4, 5, 6
2	2, 3, 4, 5, 6
3	3, 4
4	4
5	5, 6
6	6

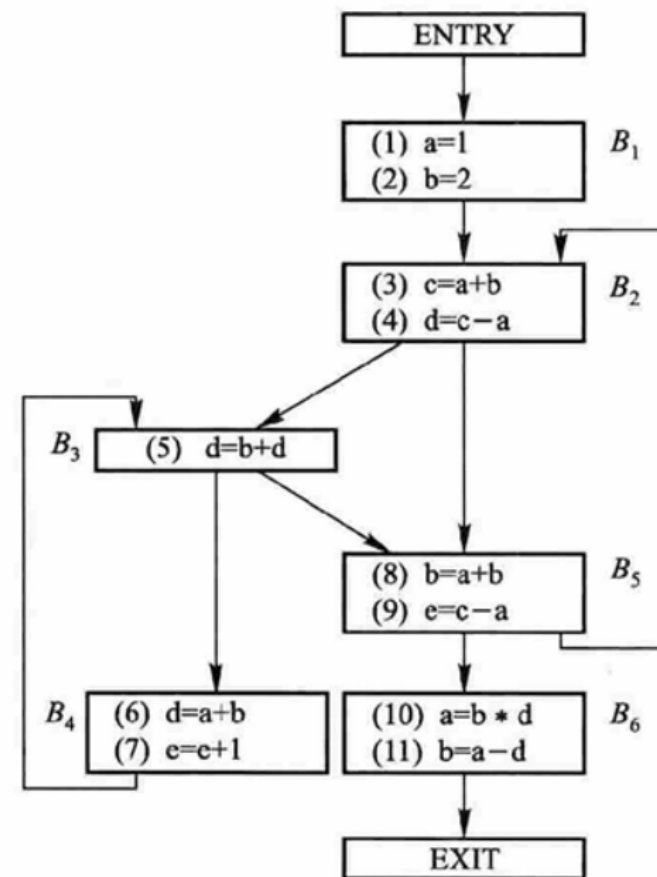


图 9.32 一个流图

9.15(b)

- 找出自然循环

自然循环: (1) 循环必须有唯一的入口结点, 称为首结点, 首节点支配该循环中所有结点
(2) 至少存在一条**回边**进入该循环首结点

回边: 流图的边 $a \rightarrow b$ 称为回边, 如果 b 支配 a

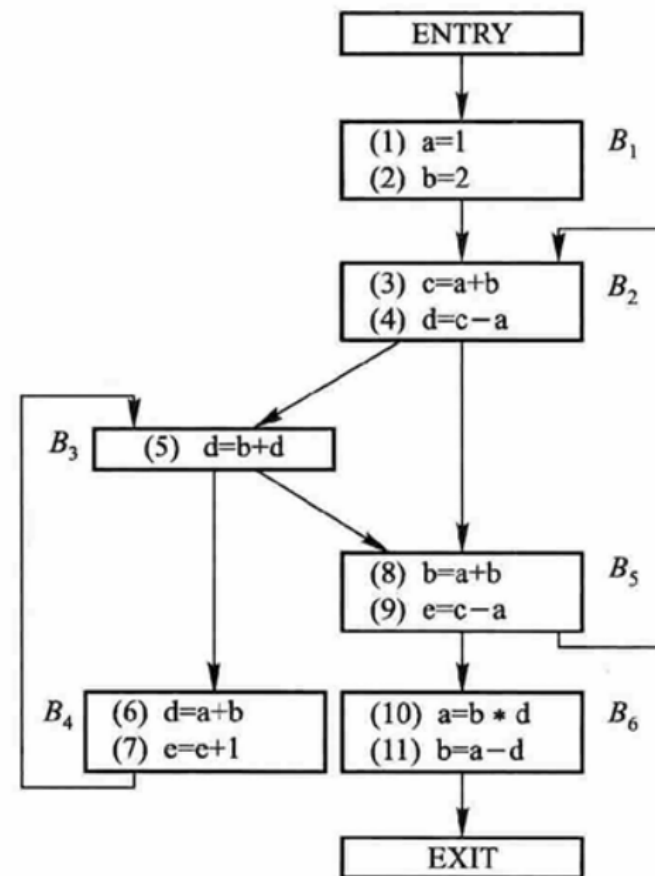


图 9.32 一个流图

9.15(b)

- 找出自然循环（具体算法在教材P315）

给定一条回边 $n \rightarrow d$ ，该边确定的自然循环是 d 加上不经过 d 能到达 n 的所有结点。结点 d 是该循环的首结点。

回边 $B_4 \rightarrow B_3$: 对应自然循环 $\{B_3, B_4\}$

回边 $B_5 \rightarrow B_2$: 对应自然循环 $\{B_2, B_3, B_4, B_5\}$

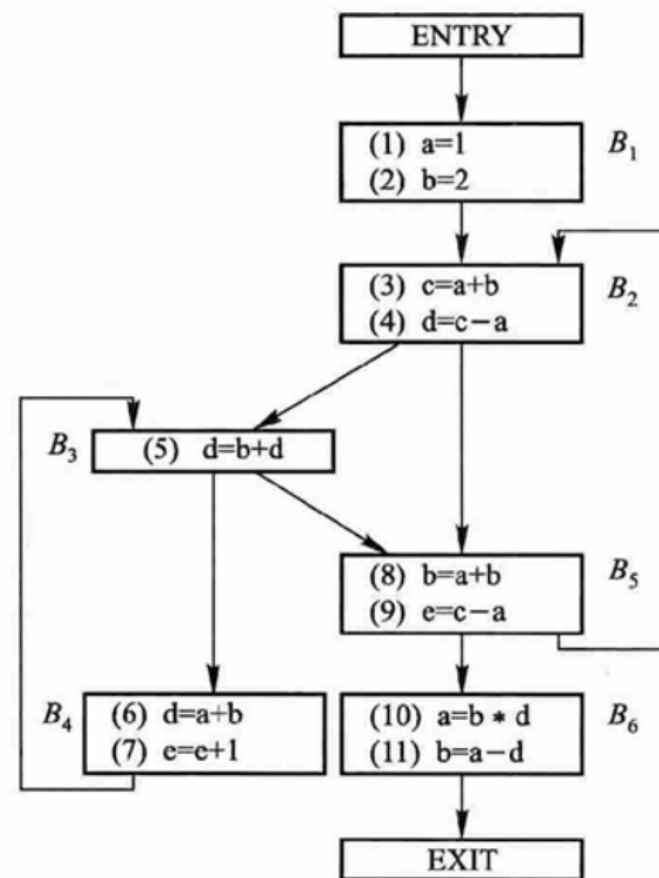


图 9.32 一个流图

9.22

9.22 请利用代码优化的思想（代码外提和强度削弱等），改写下面 C 语言程序中的循环，得到优化后的 C 语言程序。

```
int main() {
    int i,j;
    int r[20][10];
    for(i=0; i<20; i++) {
        for(j=0; j<10; j++) {
            r[i][j] = 10*i*j;
        }
    }
    return 0;
}
```

循环不变
量外提

```
int main() {
    int i,j;
    int r[20][10];
    int temp = 0;
    for(i=0; i<20; i++) {
        temp = 10 * i;
        for(j=0; j<10; j++) {
            r[i][j] = temp*j;
        }
    }
    return 0;
}
```

强度削弱

```
int main() {
    int i,j;
    int r[20][10];
    int temp = 0;
    int val = 0;
    for(i=0; i<20; i++) {
        val = 0;
        for(j=0; j<10; j++) {
            r[i][j] = val;
            val += temp;
        }
        temp += 10;
    }
    return 0;
}
```

这里的强度削弱为内层和外层循环均进行了强度削弱后的结果

9.22

在循环不变量外提之后，两次强度削弱：

```
int main() {  
    int i,j;  
    int r[20][10];  
    int temp = 0;  
    for(i=0; i<20; i++) {  
        temp = 10 * i;  
        for(j=0; j<10; j++) {  
            r[i][j] = temp*j;  
        }  
    }  
    return 0;  
}
```

第一次
强度削弱

```
int main() {  
    int i,j;  
    int r[20][10];  
    int temp = 0;  
    int val = 0;  
    for(i=0; i<20; i++) {  
        temp = 10 * i;  
        val = 0;  
        for(j=0; j<10; j++) {  
            r[i][j] = val;  
            val += temp;  
        }  
    }  
}
```

第二次
强度削弱

```
int main() {  
    int i,j;  
    int r[20][10];  
    int temp = 0;  
    int val = 0;  
    for(i=0; i<20; i++) {  
        val = 0;  
        for(j=0; j<10; j++) {  
            r[i][j] = val;  
            val += temp;  
        }  
        temp += 10;  
    }  
    return 0;  
}
```

谢谢!



中国科学技术大学
University of Science and Technology of China

